

第二章：进程管理

一、核心概念

1.1 进程（Process）

进程是操作系统资源分配的基本单位，是"正在执行的程序"的抽象。一个进程包含：

- **地址空间**：代码、数据、堆、栈的虚拟内存映射
- **执行状态**：程序计数器（PC）、寄存器、栈指针
- **资源**：打开的文件、信号处理表、进程间通信句柄
- **元数据**：PID、父进程、优先级、状态

1.2 线程（Thread）

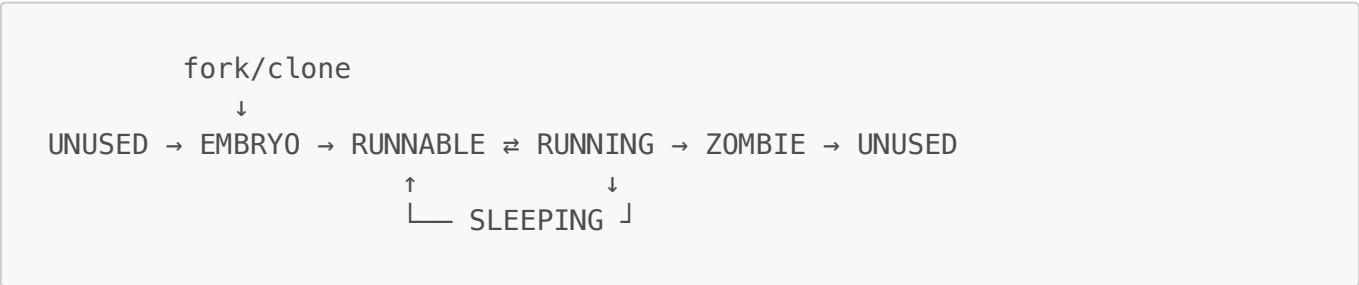
线程是 CPU 调度的基本单位。同一进程的多个线程共享地址空间和资源，但各有独立的执行状态（PC、寄存器、栈）。

进程 vs 线程的核心区别：

属性	进程	线程
地址空间	独立	共享
文件描述符	独立（fork 复制）	共享
信号处理	独立	部分共享（处理函数共享，mask 独立）
创建开销	大（复制页表、fd 表）	小（只需分配栈和 TCB）
通信	需要 IPC 机制	直接读写共享内存

1.3 进程状态机

进程在其生命周期中经历以下状态转换：



- **RUNNABLE**：就绪，等待 CPU 调度
- **RUNNING**：正在 CPU 上执行
- **SLEEPING**：阻塞等待事件（I/O、锁、信号）
- **ZOMBIE**：已退出但父进程尚未回收

1.4 上下文切换（Context Switch）

当 CPU 从一个进程/线程切换到另一个时，需要：

- 1. 保存当前执行上下文（寄存器、PC、栈指针）
- 2. 恢复目标执行上下文
- 3. 切换地址空间（页表基址寄存器）

关键洞察：上下文切换总是发生在内核态。用户态 → 内核态的切换由 trap 完成（保存所有寄存器到 trapframe），内核态之间的切换由 swtch 完成（只保存 callee-saved 寄存器，因为是普通函数调用）。

1.5 调度算法

调度器决定哪个就绪进程获得 CPU。经典算法：

算法	思想	优点	缺点
FCFS	先来先服务	简单	短任务等待长任务（护航效应）
RR	时间片轮转	公平，响应快	无优先级区分
MLFQ	多级反馈队列	兼顾交互性和吞吐量	参数调优复杂
CFS	完全公平调度	自动平衡	实现复杂（红黑树）
优先级调度	静态/动态优先级	区分重要性	可能饥饿

1.6 信号（Signal）

信号是 Unix 系统中的异步通知机制。进程可以向另一个进程发送信号，接收方可以：

- 执行默认动作（终止、忽略、停止）
- 捕获并执行自定义处理函数
- 屏蔽信号（延迟处理）

信号的关键特性：

- **异步：**信号可以在任何时刻到达
- **不排队**（传统信号）：同一信号多次发送只记录一次
- **可屏蔽：**通过 sigmask 暂时阻止信号投递

1.7 进程创建模型

Unix 系统使用 **fork + exec** 模型：

- **fork()**：创建当前进程的副本（子进程）
- **exec()**：用新程序替换当前进程的地址空间

Linux 进一步统一为 **clone()** 系统调用，通过 flags 控制父子共享的资源，从而同时支持进程创建和线程创建。

二、基本推论

推论 1: PCB/TCB 分离是线程高效共享的前提。 如果进程和线程使用同一个数据结构, 那么"线程"其实就是另一个完整的进程, 只是碰巧共享了页表。这导致进程级资源 (信号处理表、文件描述符表) 要么无法共享, 要么需要手动同步。

推论 2: 上下文切换的开销决定了调度频率的上限。 保存/恢复寄存器、刷新 TLB、切换页表都有时间开销。如果时间片太小, CPU 大部分时间花在切换上; 如果太大, 交互性下降。

推论 3: MLFQ 的核心是"用行为推断类型"。 用完时间片的进程可能是 CPU 密集型 → 降级; 主动 yield 的进程可能是 I/O 密集型 → 保持高优先级。周期性 boost 防止低优先级进程饥饿。

推论 4: 信号投递必须在安全的时刻进行。 内核不能在任意点打断进程去执行信号处理函数——那样会破坏内核的数据结构一致性。因此, 信号总是在从内核态返回用户态的边界上投递。

推论 5: fork + exec 模型看似浪费 (先复制再丢弃), 但 COW 让 fork 接近零开销。 真正的拷贝延迟到写入时才发生, 而 exec 会直接替换整个地址空间, 因此 fork 复制的那些页表项从未被实际拷贝。

三、Linux / 通用实现

3.1 Linux 的进程/线程模型

Linux 内部使用 `task_struct` 统一表示进程和线程。一个"进程"是一组共享信号处理、文件描述符表、地址空间的 `task_struct` 集合 (线程组)。

```
task_struct (thread)
├── pid           (线程 ID, 每个线程唯一)
├── tgid          (线程组 ID = 进程 PID)
├── mm_struct*    (地址空间, 同进程的线程共享)
├── files_struct* (文件描述符表, 同进程共享)
├── signal_struct* (信号处理, 进程级共享)
├── sighand_struct*
├── thread_info   (内核栈底部, 快速访问)
└── stack         (独立的内核栈)
```

clone() 的 flags 控制共享粒度:

- **CLONE_VM**: 共享地址空间 (mm_struct)
- **CLONE_FILES**: 共享文件描述符表
- **CLONE_SIGHAND**: 共享信号处理函数
- **CLONE_THREAD**: 加入父进程的线程组

`fork() = clone(SIGCHLD)` (什么都不共享) `pthread_create() = clone(CLONE_VM | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD | ...)`

3.2 Linux CFS 调度器

Linux 从 2.6.23 开始使用 **CFS (Completely Fair Scheduler)** :

- **核心思想**: 每个进程维护 **vruntime** (虚拟运行时间), CFS 总是选择 vruntime 最小的进程运行
- **数据结构**: 红黑树 (按 vruntime 排序), 选择最左节点 = O(1)

- **权重**：nice 值影响 vruntime 增长速率，nice 越低（优先级越高），vruntime 增长越慢
- **时间片**：不是固定的，由当前可运行进程数和 `sched_latency` 动态计算

$$\text{vruntime 增量} = \text{实际运行时间} \times (\text{NICE_0_WEIGHT} / \text{进程权重})$$

nice -20 的进程权重约为 nice 0 的 88x，因此 vruntime 增长极慢，获得更多 CPU 时间。

3.3 Linux 信号实现

Linux 的信号投递流程：

1. `kill(pid, sig)` → 在目标进程的 `pending` 位图中置位
2. 返回用户态路径 `exit_to_user_mode()` → 检查 `pending & ~blocked`
3. 有待处理信号 → `setup_rt_frame()` 在用户栈构造 `rt_sigframe`
4. 修改 `pt_regs`：IP = handler，SP = sigframe 顶部，返回地址 = `__restore_rt`
5. 返回用户态，执行 handler
6. handler 返回 → 执行 `__restore_rt` → `rt_sigreturn` syscall → 恢复原始寄存器

实时信号 (32-64)：可排队，携带额外数据 (`siginfo_t`)。

3.4 通用 MLFQ 设计

Solheim 和 Arpaci-Dusseau 提出的经典 MLFQ 五条规则：

1. 高优先级队列的进程优先执行
2. 同优先级的进程轮转执行
3. 新进程从最高优先级开始
4. 用完时间片的进程降级
5. 周期性地将所有进程提升到最高优先级（防饥饿）

四、RUOS 的实现

核心文件

- `src/proc/proc.h / proc.c` — struct proc、进程生命周期、scheduler
- `src/proc/mlfq.h / mlfq.c` — MLFQ 四级调度器
- `src/proc/signal.h / signal.c` — 64 路信号处理
- `src/proc/kthread.c` — 内核线程
- `src/proc/swtch.S` — 上下文切换

4.1 一体化 proc 结构

RUOS 沿用 xv6 的设计，不分离 **PCB/TCB**，所有状态都在同一个 `struct proc` 中：

身份：pid, state, name, parent, exit_signal
 执行：trapframe, context, kernel_stack
 内存：pagetable, sz, vma (mmap 链表)

文件: ofile[128], fdflags[128], cwd, cwdpath
信号: sigpending, sigmask, sigactions[64]
线程: clear_child_tid, is_kthread, kthread_fn/arg
调度: priority, mlfq (内含 level, ticks_used)
IPC: shm_attach[16], uid, gid

对比 Linux: Linux 的 `task_struct` 通过指针共享 `mm_struct`、`files_struct`、`signal_struct`，实现了真正的线程语义。RUOS 的 clone 出来的"线程"只是共享了页表的独立 proc。

优点: 代码路径简单，不用处理 PCB/TCB 之间的指针关系和生命周期管理。

缺点: 线程之间不能共享信号处理表和文件描述符表。一个线程修改 VMA 列表，另一个线程不一定能看到。这是一个已知的简化。

4.2 MLFQ 四级调度

xv6 原版是 O(n) 遍历 proc[] 找 RUNNABLE，无优先级概念。RUOS 实现了 MLFQ:

级别	时间片 (ticks)	用途
0 (最高)	2	交互型任务、刚创建的进程
1	4	
2	8	
3 (最低)	16	CPU 密集型任务

```
// mlfq_tick() - 每个时钟中断调用
p->mlfq.ticks_used++;
if (p->mlfq.ticks_used >= timeslice[p->mlfq.level]) {
    if (p->mlfq.level < MAX_LEVEL-1) p->mlfq.level++; // 降级
    p->mlfq.ticks_used = 0;
    return 1; // 需要重新调度
}

// 全局 boost: 每 100 ticks 把所有进程提升到 level 0
if (global_ticks % BOOST_INTERVAL == 0) {
    for all proc: p->mlfq.level = 0, p->mlfq.ticks_used = 0;
}
```

对比 Linux CFS:

	RUOS MLFQ	Linux CFS
数据结构	4 个优先级队列	红黑树
选择策略	最高非空队列的第一个	vruntime 最小的
时间片	固定 (2/4/8/16 ticks)	动态计算

RUOS MLFQ		Linux CFS
防饥饿	周期性 boost	vruntime 天然保证
复杂度	简单 (~100 行)	复杂 (~数千行)

设计理由：比赛要求 `sched_yield` 和动态优先级，MLFQ 用最小的代码量满足需求。

4.3 上下文切换

`swtch.S` 只保存 callee-saved 寄存器 (`s0-s11, ra, sp`)，因为 `swtch()` 被当作普通 C 函数调用，caller-saved 寄存器由编译器在调用前处理。

```
进程 A (内核态) → sched() → swtch(&A.context, &cpu.context)
→ 保存 A 的 ra/sp/s0-s11
→ 恢复 scheduler 的 ra/sp/s0-s11
→ ret 回到 scheduler() 循环
scheduler 选中进程 B → swtch(&cpu.context, &B.context)
→ 保存 scheduler 的寄存器
→ 恢复 B 的寄存器
→ ret 回到 B 之前调用 sched() 的地方
```

对比 Linux：Linux 的 `__switch_to()` 也只保存 callee-saved 寄存器，但还需要处理 FPU 状态、TLS 段寄存器、内核栈切换等。RUOS 不需要处理 FPU（没有浮点支持）。

4.4 fork vs clone_fork

fork（传统）：

- COW 复制页表
- 复制文件描述符表
- 子进程独立运行

clone_fork（线程式）：

```
if (flags & CLONE_VM) {
    np->pagetable = p->pagetable; // 共享页表
    np->sz = p->sz;
} else {
    uvmcopy(p->pagetable, np->pagetable, p->sz); // COW 复制
}
```

支持的 clone flags：

- `CLONE_VM`：共享页表
- `CLONE_SETTLS`：设置线程局部存储 (tp 寄存器)
- `CLONE_CHILD_CLEAR_TID`：线程退出时清零 `clear_child_tid` 并唤醒 futex
- `CLONE_CHILD_SETTID`：创建时写入子线程 tid

4.5 信号处理

RUOS 模仿 Linux 的 64 信号模型。

数据结构：

- **sigpending**: uint64 位图
- **sigmask**: uint64 位图
- **sigactions[64]**: 每个信号的 handler/flags/mask

投递时机：从内核态返回用户态时（**usertrapret** 路径），检查 **sigpending & ~sigmask**。

信号帧机制：

1. 在用户栈上构造 **RtSigframe**（保存所有用户寄存器 + sigmask）
2. 设 **epc** = handler 地址, **ra** = restorer 地址
3. 返回用户态执行 handler
4. handler 返回 → restorer → **rt_sigreturn** syscall
5. 内核从 sigframe 恢复原始寄存器和 sigmask

对比 Linux：流程完全一致，RUOS 忠实地复现了 Linux 的信号帧模型。区别在于 Linux 支持实时信号排队、信号的线程组投递等高级特性。

4.6 八股 / 面试高频问题

Q: 进程和线程的区别？ A: 进程是资源分配的单位（地址空间、文件描述符、信号处理），线程是调度的单位（PC、寄存器、栈）。同一进程的线程共享地址空间和资源。Linux 中线程和进程都用 **task_struct** 表示，区别在于 **clone flags** 决定共享什么。

Q: 进程有哪些状态？状态转换条件？ A: 创建→就绪（**RUNNABLE**）→运行（**RUNNING**）→阻塞（**SLEEPING**）→就绪。运行→僵尸（**ZOMBIE**，已退出等待父进程回收）。关键转换：时间片用完/yield→就绪；I/O等待/锁等待→阻塞；I/O完成/wakeup→就绪；exit→僵尸；父进程wait→回收。

Q: 什么是上下文切换？开销有多大？ A: 保存当前进程寄存器、切换页表（**satp/CR3**）、恢复目标进程寄存器。开销包括：寄存器保存/恢复（~几十 ns）、TLB 刷新（可能 ~几百 ns 到几 us）、cache 冷启动（间接开销）。Linux 典型上下文切换 ~1-10 us。

Q: CFS 调度器的核心思想？ A: 完全公平——每个进程维护虚拟运行时间（**vruntime**），调度器总是选 **vruntime** 最小的进程。**nice** 值影响 **vruntime** 增长速率。用红黑树存储，选择最左节点 **O(1)**。

Q: 什么是僵尸进程？什么是孤儿进程？ A: 僵尸进程：已 **exit** 但父进程未 **wait** 回收，**task_struct** 仍占用。孤儿进程：父进程先退出，被 **init(PID 1)** 收养。大量僵尸进程会耗尽 **PID**。

Q: fork 的 COW 是怎么工作的？ A: **fork** 时不复制物理页，父子共享且标记只读。任一方写入时触发 **Page Fault**，内核复制该页给写入方。如果子进程立即 **exec**，复制的页表项全部丢弃，真正的物理拷贝为零。

Q: 信号的投递时机是什么？ A: 从内核态返回用户态时（系统调用返回/中断返回），检查 **pending & ~blocked**。在用户栈构造 **sigframe**，修改 **PC** 指向 handler。handler 返回后通过 **sigreturn** 恢复原始上下文。

4.7 内核线程

用于网络协议栈的后台收包：

```
kthread_create(ethernet_ii_recv, NULL, "net_rx");
```

分配一个 proc 槽，标记 `is_kthread=1`，不创建用户页表。`context.ra` 设为 `kthread_trampoline`，调用 `kthread_fn(kthread_arg)` 然后 `kthread_exit()`。

对比 Linux：Linux 的 `kthread_create()` 创建的内核线程也是 `task_struct`，共享内核地址空间（`mm = NULL`），思路一致。

五、项目贡献亮点

- 实现了 MLFQ 四级调度器（~100 行），替代 xv6 的 O(n) 朴素调度
- 实现了 64 路 POSIX 信号 + sigframe 机制，与 Linux 信号投递流程完全一致
- 实现了 clone 系统调用，支持 CLONE_VM/SETTTL/CHILD_CLEARID 等线程创建 flag
- 内核线程用于网络协议栈后台收包，理解内核线程与用户进程的区别

六、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
进程/线程模型	不分离，一体化 proc	task_struct + 指针共享	简单但线程语义不完整
调度算法	MLFQ 4 级	CFS 红黑树	代码量小，效果够用
信号模型	64 信号 + sigframe	完整 POSIX 信号	Linux 兼容，通过测试
进程数上限	64 (NPROC)	动态分配 (slab)	静态数组，简单但有上限
文件描述符	128 (NOFILE)	动态扩展 (fdtable)	固定大小，BusyBox 够用
上下文切换	14 寄存器	callee-saved + FPU + TLS	不支持浮点，更简单